Final Project Report

CSE115 — Group 09

Members: Niladri Das Chanchal · Eashfee Ahmed · Gourav Saha · Md. Jawad Muhib

1. Introduction

Background

Small shops need a reliable way to track available products, remaining stock, and the effect of each sale. A console application is a manageable way to demonstrate the core workflow without a database or graphical interface.

Project overview

Inventory & Sales Manager is a menu-driven C program prepared for CSE115 Group 09. It manages product records, changes stock, records sales, creates basic reports, and saves records in CSV files.

Problem statement

Manual tracking can lead to duplicate records, inaccurate quantities, and sales being accepted when stock is insufficient. The program uses unique product IDs, input validation, and stock checks to reduce these errors.

Objectives

Practise fundamental C programming; maintain searchable products; prevent duplicate product IDs; validate sales before changing stock; calculate totals; show sales and low-stock summaries; and persist records between runs.

2. Scope and Requirements

Functional requirements

Add products, display inventory, search by ID, update product details, restock, record sales, display sales history, calculate a summary, report low stock, save records, and exit after saving.

Data requirements

A Product stores ID, name, category, unit price, quantity, and low-stock threshold. A Sale stores sale ID, product ID, product name at the time of sale, quantity, unit price, total, and date text.

Input rules

Product IDs must be positive and unique. Quantities and thresholds must be non-negative; sale quantity must be positive. Prices must be non-negative. Empty names/categories and comma-containing product fields are rejected.

Non-functional requirements

The console interface should be understandable to a beginner, input errors should not crash the program, the source should compile with a standard C compiler, and build/test documentation should be included.

Out of scope

Authentication, multi-user access, a database server, barcode scanning, payment processing, tax/discount processing, a graphical interface, and production-grade auditing are not included.

3. Design and Architecture

Program structure

The implementation is kept in main.c so it is straightforward to compile and review. Functions separate input handling, inventory operations, sales operations, reports, and file persistence.

Product structure

Product groups the product ID, name, category, price, stock quantity, and low-stock threshold. One shared array is the inventory source for all operations.

Sale structure

Sale records the transaction ID, product identity and name, quantity, unit price, total, and date text. The sales array supports history and summary calculations.

Menu flow

At startup the program attempts to load saved files. It then displays a menu, validates the selected option, calls the relevant function, and returns to the menu. Exit attempts to save both data sets.

Integration principle

All features use the same shared arrays and structures. This avoids competing inventory copies and keeps sale records connected to stock changes.

4. Implementation

Inventory management

Adding a product checks capacity and ID uniqueness. Product name and category must be non-empty and comma-free for simple CSV storage. Display/search show stored values; update changes a selected field; restocking adds a positive amount after checking bounds.

Sales processing

The program finds the product, reads a positive quantity, and compares it with available stock. An oversell is cancelled without changing stock or sales history. Otherwise, the program calculates the total, collects date text, adds the sale record, and reduces stock.

Reports

Sales history lists transactions. The summary calculates transaction count, units sold, and recorded revenue from the sales array. The low-stock report shows products with quantity less than or equal to the configured threshold.

File handling

inventory.csv and sales.csv store records. The program writes a header and one record per line, and attempts to load records at startup. Missing files are acceptable on a first run. Files are created in the current working directory.

Input validation

Numeric input is parsed from a line and checked for a valid range. This reduces common input-buffer problems and provides a clear retry prompt when input is invalid.

5. Testing and Verification

Test approach

Testing should cover normal operations, invalid input, boundaries, inventory/sales consistency, and persistence. Execute tests listed in tests/manual_test_cases.csv and record actual outcomes.

Core scenarios

Add a valid product; reject a duplicate ID; search existing and missing IDs; update/restock stock; record a sale within stock; reject an oversell; check summary totals; trigger a low-stock warning; save and reload records.

Expected invariants

Product IDs are unique. A successful sale decreases stock by the sold quantity. A rejected sale changes neither stock nor sales count. The summary agrees with stored sale records. Saved data should remain available after restarting.

Status and evidence

No test outcome is asserted in this report. The team must run each test and fill in actual result, date, and tester fields. A test should be marked passed only after observing the expected behavior.

6. Team Responsibilities and Workflow

Niladri Das Chanchal — Project Lead & Integration

Coordinate scope and shared interfaces, integrate the menu and program flow, review changes, proofread documentation, and ensure the presentation matches the submitted code.

Eashfee Ahmed — Inventory Management

Own and explain product creation, listing, searching, updating, restocking, and product validation. Demonstrate duplicate-ID rejection and stock changes.

Md. Jawad Muhib — Sales Management

Own and explain sale validation, transaction totals, stock deduction, history, and summary. Demonstrate both a valid sale and an oversell rejection.

Gourav Saha — Persistence, Testing & QA

Own and explain CSV save/load, maintain the test plan, run regression checks, review edge cases, and document build/run steps.

Collaboration

Members should review shared code, understand the complete program flow, communicate interface changes before merging, and make genuine commits for work actually performed.

7. Limitations, Risks and Future Work

Current limitations

The program uses fixed capacities, is single-user and console-based, uses basic CSV parsing, and stores the date as text without validating calendar correctness. Names/categories cannot contain commas.

Risks

Risks include inconsistent edits to shared structures, malformed manually edited CSV files, missed edge cases, untested file permissions, and a demo that differs from submitted code. Shared ownership and testing reduce these risks.

Future improvements

Possible future work includes modular source/header files, stronger CSV parsing, date validation, more detailed reports, configurable paths, safer file updates, product deletion, and automated tests if permitted by the course.

Reliability note

This is an educational demonstration, not a production inventory or payment system. It has no authentication, encryption, concurrent-access protection, or formal audit trail.

8. Conclusion and References

Conclusion

Inventory & Sales Manager connects inventory management, sales processing, reporting, and file handling in a beginner-level C application. Its key consistency rule is that a sale must be validated before stock changes. The team should run and document all tests before presenting.

References and AI assistance

Add exact external references, tutorial videos, and AI tools actually used to support/REFERENCES.md. Do not list sources that were not used. Members must understand and take responsibility for the submitted code.

Submission readiness

The final repository should include source code, README, requirements, data/support folders, reports, presentations, demo script/video, and test matrix. Upload required files before the instructor's stated deadline and verify that the demo matches the final build.